```
terraform-infra/
│
├── modules/
│   ├── vpc/
│   │   ├── main.tf
│   │   ├── variables.tf --> Declare variable schema
│   │   └── outputs.tf
│   │
│   ├── ecs/
│
├── environments/
│   ├── dev/
│   │   ├── main.tf
│   │   ├── backend.tf
│   │   ├── terraform.tfvars --> Assign value to variable
│   │   ├── providers.tf
```


In your **Terraform directory structure**, the **VPC module is reusable infrastructure code**, and the **dev environment calls that module and passes values to it**.

Let's walk through **how the `dev` environment uses the `vpc` module**.

1️⃣ VPC Module (Reusable Code)

Path:

```
modules/vpc/
```

This module contains the **actual AWS resources**.

modules/vpc/main.tf

```hcl
resource "aws_vpc" "main" {
  cidr_block = var.vpc_cidr

  tags = {
    Name = var.vpc_name
  }
}
```

modules/vpc/variables.tf

Here we **define variables (schema)**.

```hcl
variable "vpc_cidr" {
  description = "CIDR block for VPC"
  type        = string
}

variable "vpc_name" {
  description = "Name of the VPC"
  type        = string
}
```

modules/vpc/outputs.tf

Expose outputs so other modules can use them.

```hcl
output "vpc_id" {
  value = aws_vpc.main.id
}
```

2️⃣ Dev Environment Calls the Module

Path:

```
environments/dev/main.tf
```

Here we **call the module**.

environments/dev/main.tf

```hcl
module "vpc" {

  source = "../../modules/vpc"

  vpc_cidr = var.vpc_cidr
  vpc_name = var.vpc_name

}
```

Explanation:

- `source` → tells Terraform where the module exists
- Variables are passed to the module

3️⃣ Dev Variables Definition

Path:

```
environments/dev/providers.tf
```

```hcl
provider "aws" {
  region = "ap-south-1"
}
```

environments/dev/variables.tf (optional but recommended)

```hcl
variable "vpc_cidr" {
  type = string
}

variable "vpc_name" {
  type = string
}
```

4️⃣ Assign Values in terraform.tfvars

Path:

```
environments/dev/terraform.tfvars
```

This is where **actual environment-specific values go**.

```hcl
vpc_cidr = "10.0.0.0/16"
vpc_name = "dev-vpc"
```

5️⃣ Backend Configuration

environments/dev/backend.tf

```hcl
terraform {
  backend "s3" {
    bucket = "terraform-state-dev"
    key    = "vpc/terraform.tfstate"
    region = "ap-south-1"
  }
}
```

This keeps **state separate per environment**.

6️⃣ Execution Flow

When you run:

```
cd environments/dev
terraform init
terraform plan
terraform apply
```

Terraform will:

1️⃣ Load `dev/main.tf`
2️⃣ See **module "vpc"**
3️⃣ Go to

```
../../modules/vpc
```

4️⃣ Read:

- main.tf
- variables.tf
- outputs.tf

5️⃣ Inject values from

```
terraform.tfvars
```

6️⃣ Create **Dev VPC**

7️⃣ Architecture Flow

```
terraform apply (dev)
        |
        ▼

environments/dev/main.tf
        |
        ▼

calls module

modules/vpc/main.tf
        |
        ▼

Creates AWS VPC
```

8️⃣ Why This Structure Is Used (Real Companies)

Benefits:

✅ Reusable modules
✅ Same module used for **dev / stage / prod**
✅ Only **terraform.tfvars changes**
✅ Clean separation of environments
✅ Easy CI/CD pipelines

Example:

```
dev   → 10.0.0.0/16
stage → 10.1.0.0/16
prod  → 10.2.0.0/16
```

But **same VPC module**.

💡 **Important Interview Point**

Companies design Terraform like this: